

Rather than trying to make one flow for every capability, I'd make roughly **10–12 major business flows**.

1. Create → Collaborate → Publish a Document

This is probably the platform's central flow.

```
graph TD; A[User enters Project] --> B[Creates Document]; B --> C[Document inherits Project permissions]; C --> D[Adds content / diagrams / code / tables]; D --> E[Invites or shares with collaborators]; E --> F[Multiple users edit simultaneously]; F --> G[Presence / cursors show active collaborators]; G --> H[Collaborators leave inline comments]; H --> I[Comments are resolved]; I --> J[Author reaches milestone]; J --> K[Publishes / names version]; K --> L[Document becomes authoritative knowledge];
```

This combines:

- Authoring
- Templates
- Real-time collaboration
- Presence
- Comments
- Mentions

- Lifecycle
- Version milestones

It's much more meaningful than listing those as disconnected features.

2. Review & Discussion Flow

This deserves its own flow because commenting isn't just a feature; it's a **review workflow**.

```
User opens document
      ↓
Highlights specific content
      ↓
Creates comment
      ↓
@mentions teammate
      ↓
Mentioned user receives notification
      ↓
User replies in thread
      ↓
Author makes requested change
      ↓
Reviewer confirms
      ↓
Thread resolved
```

This directly corresponds to the inline discussion requirements in the BRD.

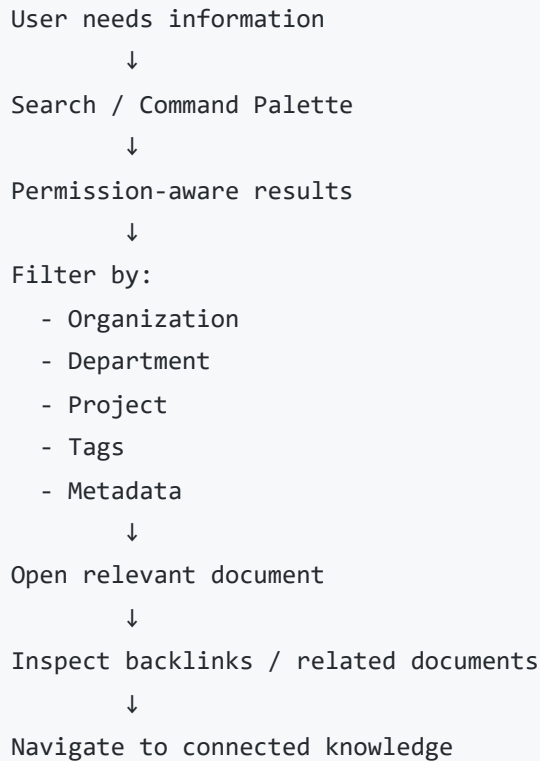
You could later extend it to:

Draft → Review → Changes Requested → Approved → Published

if you want the lifecycle to become more formal.

3. Discover Knowledge

This should be a major flow because your product isn't merely a document editor.



This represents the **Discovery & Navigation** side much better than simply saying "Unified Search."

Your BRD already defines search across the repository while respecting access boundaries.

4. Explore Knowledge Graph

This is different from normal search.

A user isn't necessarily looking for a particular document. They're trying to understand **how knowledge is connected**.

```
graph TD; A[User opens Document A] --> B[Views backlinks]; B --> C[Sees related documents]; C --> D[Opens Local Graph]; D --> E[Expands 1-hop / 2-hop relationships]; E --> F[Traverses A → B → C]; F --> G[Identifies relationship types]; G --> H[Finds relevant knowledge cluster];
```

This is directly supported by the graph requirements: multi-hop traversal, shortest pathways, local subgraphs, and semantic relationships.

5. Impact Analysis Before Changing Knowledge

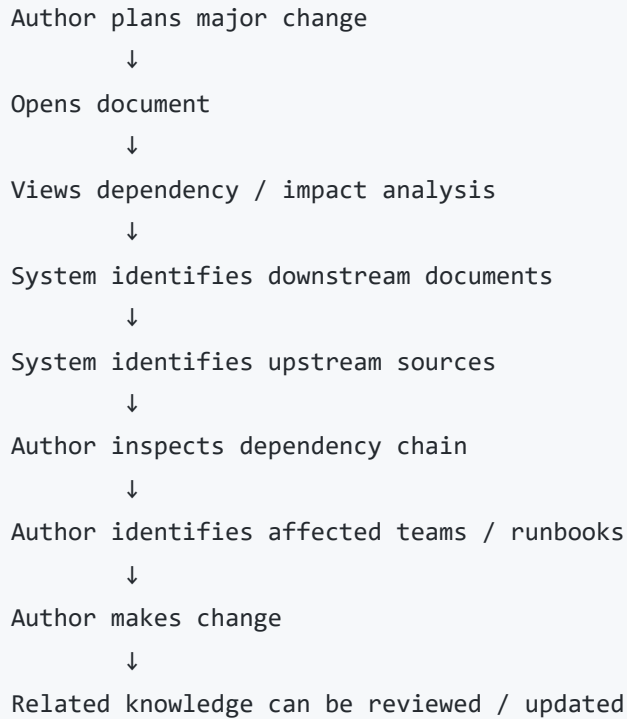
This is one of your strongest flows and deserves its own BRD section.

The important thing isn't "we have a graph."

It's:

"Before I change this document, show me what could be affected."

Flow:

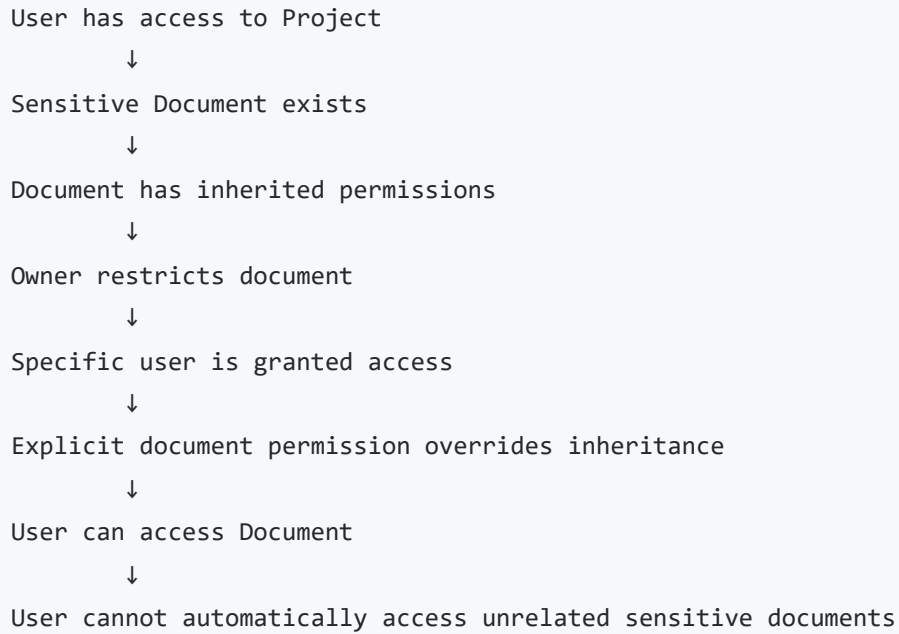


This maps directly to your stated goal of preventing dependency blindspots.

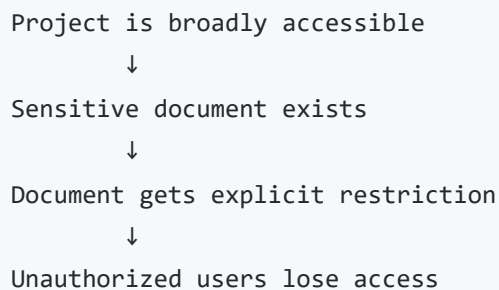
I'd actually give this flow more prominence than the generic "graph explorer."

6. Share a Restricted Document Without Opening the Whole Project

This is the defining access-control flow of your system.



And the reverse:

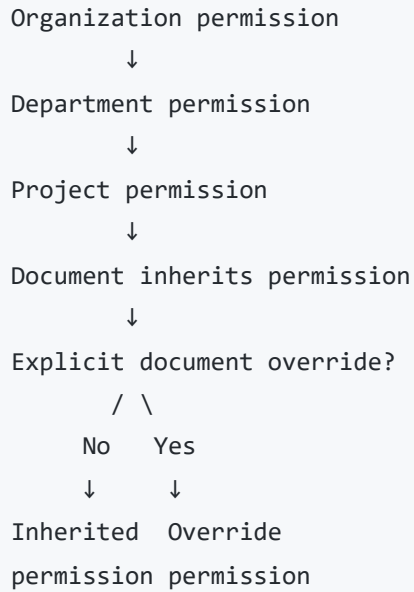


Your BRD explicitly frames this around solving the "binary access dilemma."

This should absolutely be a first-class flow.

7. Permission Inheritance & Override Resolution

I'd actually separate this from the previous business scenario because it's a different conceptual flow.



Then:



This makes the BRD's permission model understandable without getting into implementation details.

The existing BRD already specifies inherited defaults, explicit overrides, least privilege, and role tiers.

8. Version → Compare → Restore

Another very natural business flow:

Document is edited continuously



Milestone reached



Named snapshot created



More changes occur



Something needs investigation



Open version history



Select Version A + Version B



View visual diff



Identify problematic change



Restore previous version



New current state created



History remains intact

The important concept here is **non-destructive restore**.

You're not saying:

"Replace the database row with old data."

You're saying:

"Make an old state the current state while preserving the historical record."

That's a meaningful business behavior.

The BRD explicitly calls for historical revisions, visual comparison, and one-click restoration.

9. Knowledge Health / Orphan Detection

This is another flow that is currently buried inside your capability tree.

```
Document exists
  ↓
Relationships continuously analyzed
  ↓
Document has no meaningful connections
  ↓
System identifies orphan
  ↓
Owner / relevant team discovers orphan
  ↓
User either:
  ├── Links it to relevant knowledge
  ├── Updates / replaces it
  └── Archives it
```

And separately:

```
Document renamed / moved
  ↓
Existing references detected
  ↓
References automatically preserved / updated
  ↓
No broken knowledge paths
```

These are two sides of **Knowledge Health**.

Your BRD already explicitly calls out both dead-link prevention and orphan detection.

10. Draft → Publish → Deprecate Knowledge

I think this is currently underrepresented in your capability map.

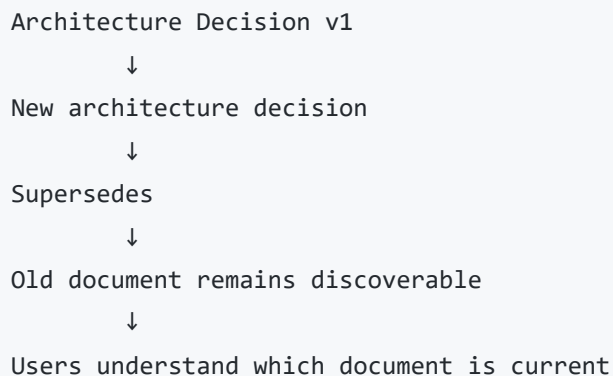
You have "Draft vs Published Milestones," but the business flow could be much clearer:



And this connects beautifully with your semantic relationship:

Supersedes / Replaces

So you can have:



This is more useful than merely having "Draft vs Published" as a bullet.

11. Audit / Investigation Flow

This is important for the governance side.

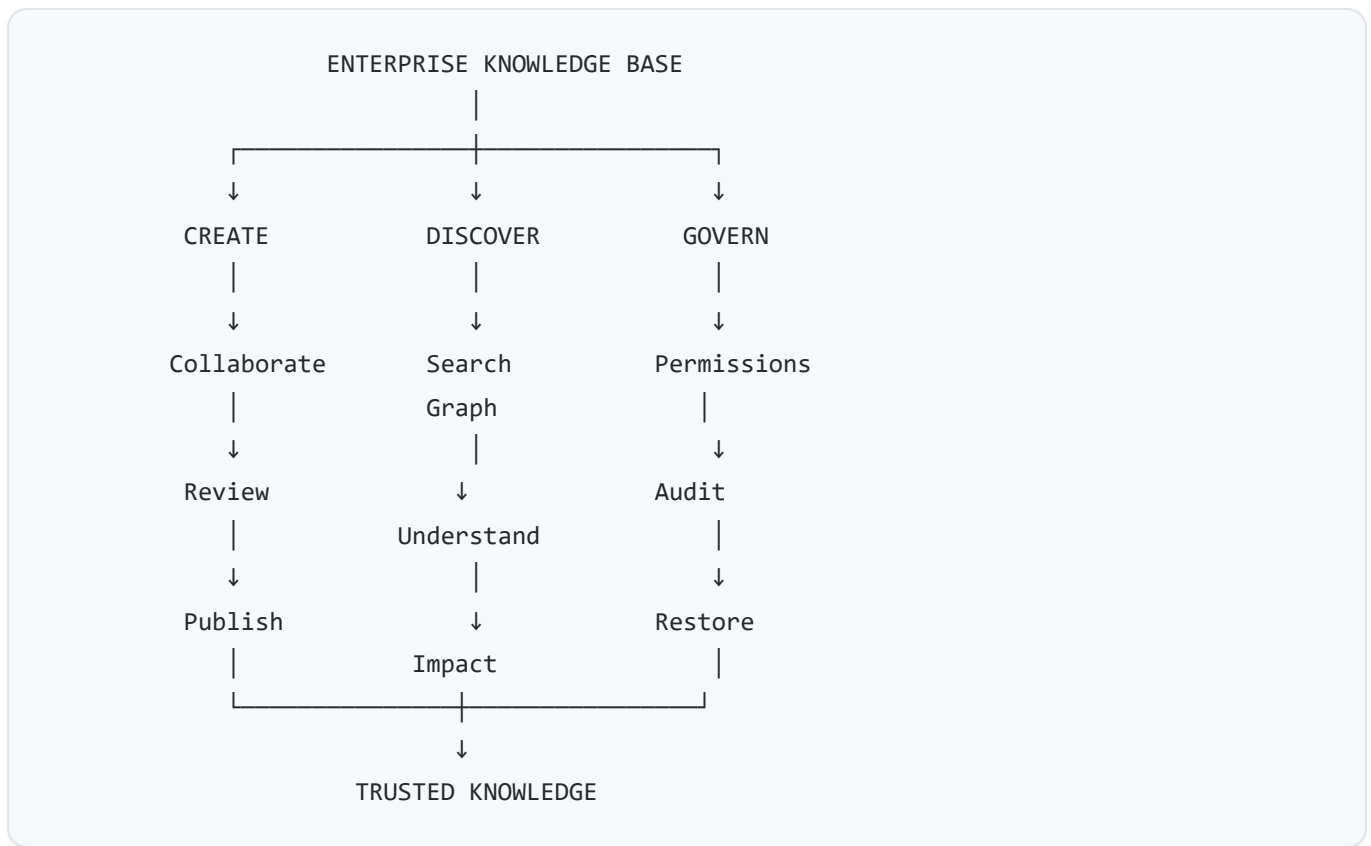
```
Security / Admin investigation
    ↓
Select user / document / period
    ↓
Inspect activity trail
    ↓
See:
- Content changes
- Permission changes
- Deletions
- Lifecycle events
- Access activity
    ↓
Trace event history
    ↓
Determine what changed / who performed it
```

The BRD explicitly positions auditability around content and access changes.

12. "Find → Understand → Act" could be your overarching flow

There's actually a nice way to tie the whole product together.

Instead of thinking of 8 isolated capability branches, your product can be understood as a few **major** journeys:



That is much closer to how I'd expect a BRD to communicate the product.

Your capability map could then become this

I wouldn't necessarily throw away your current 8 capabilities. I'd keep them as the **capability architecture**, but introduce a separate **Business/User Flows** section.

Something like:

Core Business Flows

Flow	Primary capabilities
BF-01: Create & Collaborate on Knowledge	Authoring, co-editing, presence
BF-02: Review & Discuss Knowledge	Comments, mentions, notifications
BF-03: Discover Knowledge	Search, tags, command palette
BF-04: Traverse Connected Knowledge	Backlinks, graph, relationships
BF-05: Analyze Knowledge Impact	Multi-hop traversal, dependencies
BF-06: Share Knowledge Securely	RBAC, inheritance, overrides
BF-07: Manage Knowledge Lifecycle	Draft, publish, supersede
BF-08: Compare & Restore Knowledge	Versions, diffs, rollback
BF-09: Maintain Knowledge Health	Dead links, orphan detection
BF-10: Investigate Activity & Changes	Audit trail

Then your **Capability Architecture** answers:

What can the platform do?

while your **Business Flows** answer:

How do users actually use those capabilities to accomplish something?

And that's the distinction I think you're feeling is missing.

One other important thing

I'd **not** make every tiny feature its own flow. For example:

"User adds a tag"

doesn't deserve a BRD flow.

But:

"User searches for knowledge → filters by tags → opens document → traverses related knowledge"

does.

Likewise, "live cursor" isn't a flow. It's part of **BF-01: Create & Collaborate on Knowledge**.

That gives you a BRD that feels like a **product specification rather than a feature inventory**.